

Parcial # 3

Johan Sebastian Jaimes Burbano

Alejandra Mayorga Barboza

Diseño de Patrones de software

Eliecer Montero Ojeda

FACULTAD CIENCIAS E INGENIERIAS UNIDADES

TECNOLOGICAS DE SANTANDER

BUCARAMANGA – SANTANDER

2026

Informe

En la actualidad, los usuarios buscan plataformas de entretenimiento que no solo les permitan visualizar contenido multimedia, sino también disfrutar de una experiencia personalizada, segura e interactiva. En respuesta a esta necesidad surge **StreamVault**, una plataforma de streaming diseñada para ofrecer a sus usuarios herramientas innovadoras que facilitan la organización, descubrimiento y gestión de contenido digital.

StreamVault permite crear bibliotecas y playlists personalizadas, recibir recomendaciones inteligentes basadas en los intereses de cada usuario, acceder a beneficios exclusivos mediante planes Premium, descargar contenido para visualizarlo sin conexión y participar en eventos especiales dentro de la plataforma. Además, incorpora funciones de control parental, sistemas de logros y mecanismos de recuperación de información que mejoran la seguridad y la experiencia de uso.

Gracias a estas características, StreamVault se posiciona como una solución moderna y competitiva dentro del mercado del streaming, enfocada en brindar comodidad, personalización y valor agregado tanto a usuarios ocasionales como a creadores y consumidores frecuentes de contenido multimedia.

Objetivo General

Desarrollar e implementar la plataforma de streaming StreamVault, ofreciendo una experiencia de entretenimiento personalizada y segura que permita a los usuarios organizar, descubrir, visualizar y gestionar contenido multimedia mediante herramientas avanzadas como bibliotecas personales, recomendaciones inteligentes, descargas premium, control parental y eventos exclusivos, aumentando el valor percibido y la satisfacción del usuario.

Objetivos Específicos

1. Facilitar la organización personalizada del contenido multimedia.

Permitir que los usuarios creen bibliotecas y playlists personalizadas donde puedan guardar, clasificar y administrar sus videos favoritos, manteniendo un acceso rápido y organizado a su contenido de interés.

2. Mejorar el descubrimiento de contenido relevante para cada usuario.

Ofrecer recomendaciones inteligentes basadas en las preferencias y categorías visualizadas por el usuario, ayudándolo a encontrar contenido de interés de manera rápida y personalizada.

3. Proporcionar beneficios exclusivos que incentiven la suscripción Premium.

Brindar acceso a funciones avanzadas como reproducción en alta definición, descargas temporales de contenido y participación en eventos exclusivos para usuarios Premium, generando una propuesta de valor diferenciadora.

4. Garantizar una experiencia segura para toda la familia.

Incorporar herramientas de control parental mediante PIN para restringir el acceso a contenido sensible, permitiendo que padres y tutores gestionen de forma segura la experiencia de visualización.

5. Incrementar la interacción y fidelización de los usuarios.

Implementar sistemas de logros, medallas, estadísticas de actividad y eventos especiales que motiven la participación continua y mejoren la experiencia de uso dentro de la plataforma.

6. Evitar la pérdida de información y mejorar la comodidad del usuario.

Ofrecer mecanismos de recuperación de cambios, guardado automático de borradores y restauración de contenido para que los usuarios puedan continuar sus actividades sin perder información importante.

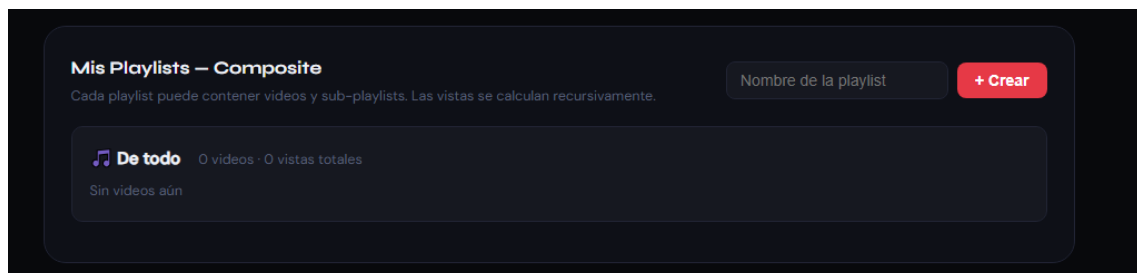
Cumplimiento de Objetivos

Objetivo 1 - Organización de contenido

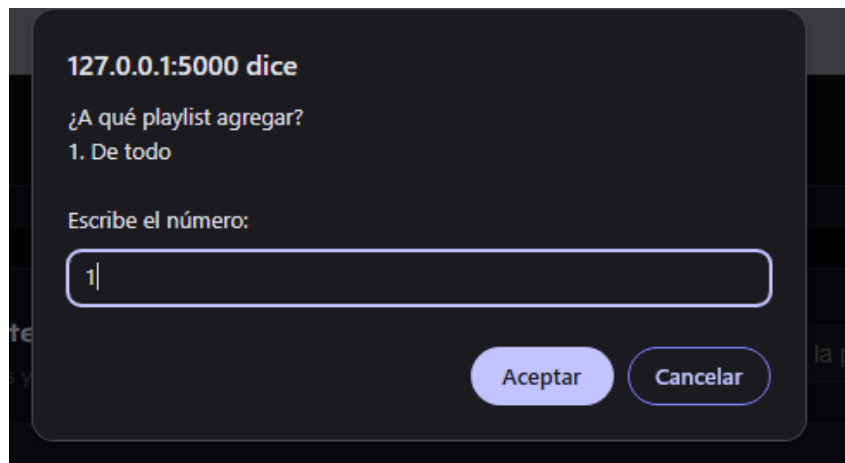
Composite — Playlists /library

StreamVault permite organizar videos mediante bibliotecas y playlists personalizadas, facilitando la gestión del contenido favorito. Los usuarios pueden agrupar videos según sus gustos, intereses o temáticas específicas, creando una experiencia totalmente personalizada y organizada.

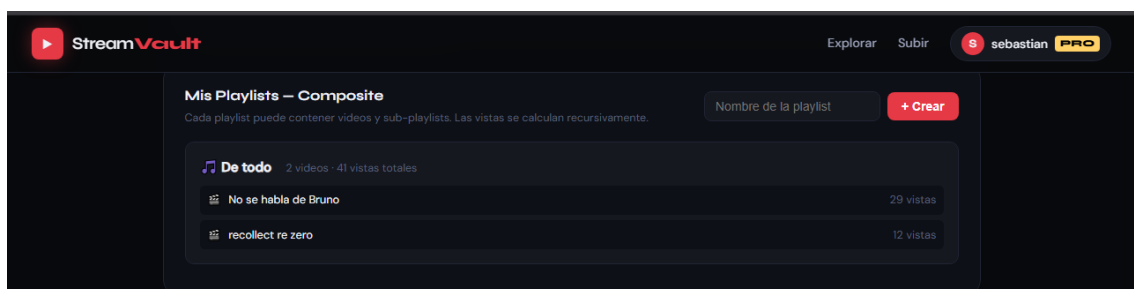
Se crea la playlists



Añadimos contenido



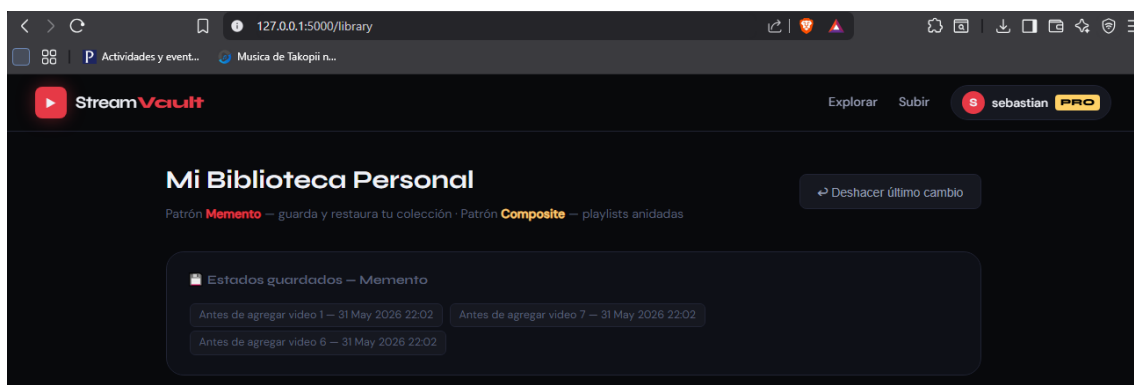
Se ven reflejado el contenido guardado y suma las vistas totales de los videos

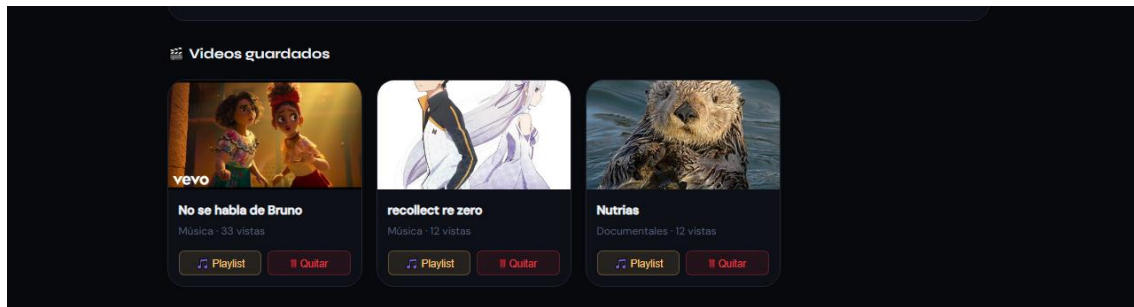


Memento — Biblioteca personal

La plataforma protege el contenido guardado por los usuarios. Si un video es eliminado accidentalmente de una biblioteca o playlist, StreamVault permite recuperar el estado anterior, evitando pérdidas de información y ofreciendo una experiencia más segura.

Junto con composite estos se complementan para la biblioteca personal de videos guardados



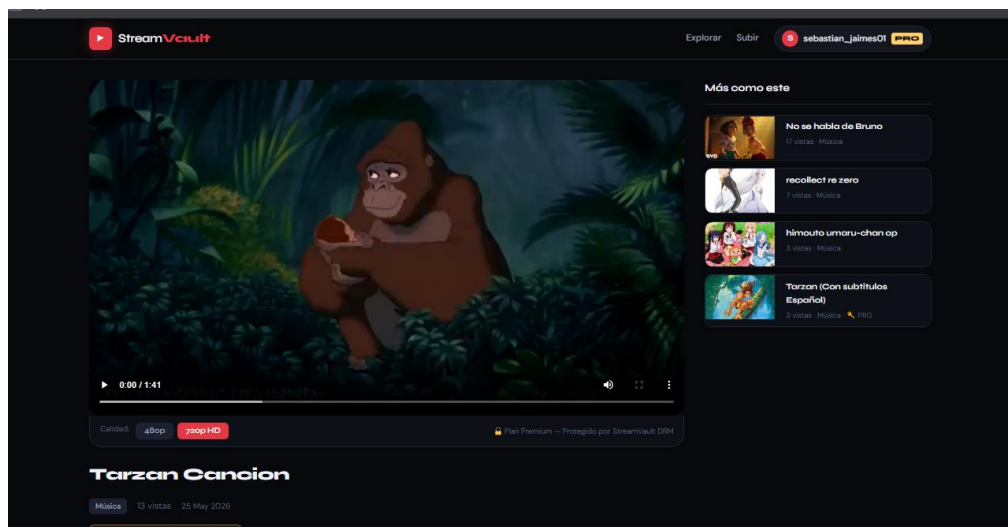


Objetivo 2 - Recomendaciones

Patrón Strategy — Recomendaciones de videos

Encontrar contenido de interés nunca había sido tan fácil. StreamVault analiza las preferencias y categorías favoritas de cada usuario para ofrecer recomendaciones personalizadas, permitiendo descubrir nuevos videos y mejorar significativamente la experiencia de entretenimiento.

Al estar viendo un video te recomienda más de dicha categoría

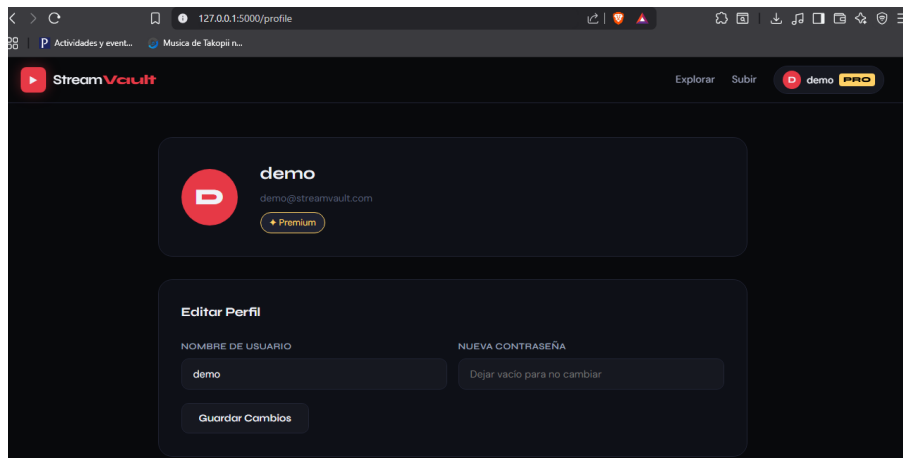


Objetivo 3 - Beneficios Premium

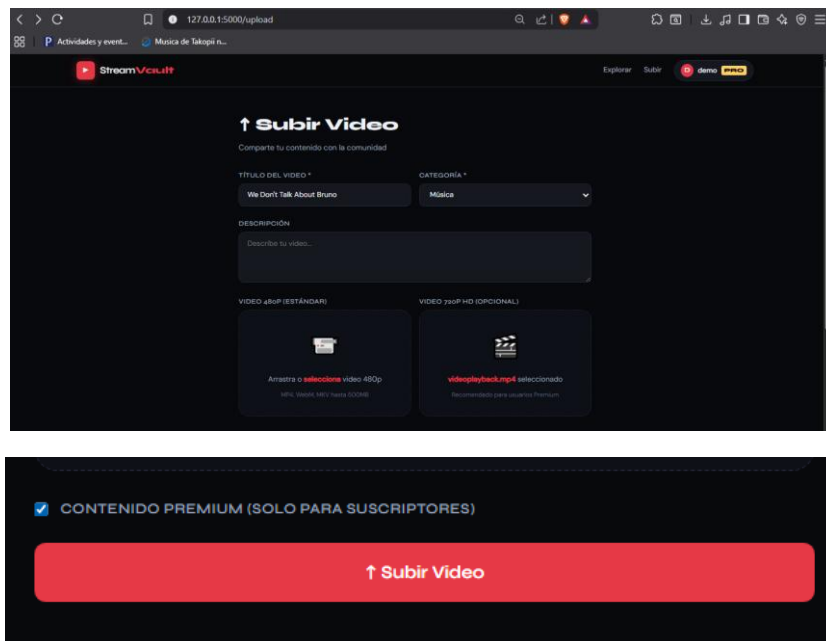
Abstract Factory – Premium y free

Gracias a StreamVault, cada usuario puede disfrutar de una experiencia adaptada a sus necesidades. Los usuarios Premium obtienen acceso a contenido en alta definición, mayores opciones de configuración y funcionalidades exclusivas, mientras que los usuarios Free pueden acceder a la plataforma y descubrir todo su potencial. Esta

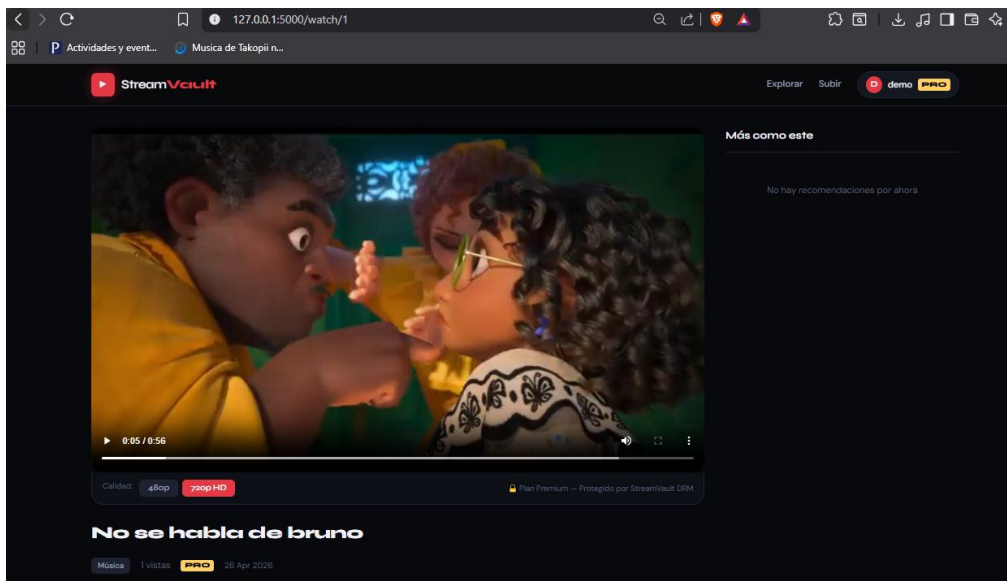
diferenciación permite ofrecer un servicio escalable y atractivo para distintos tipos de público.



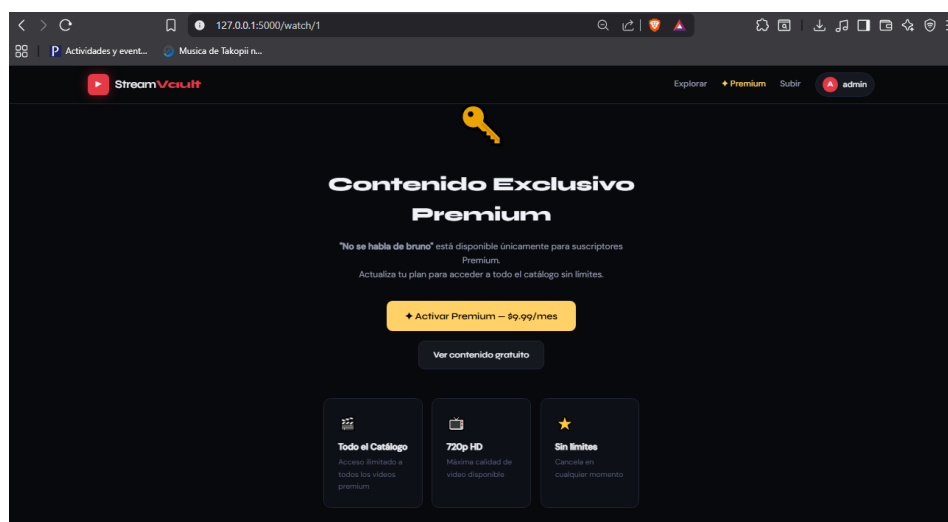
Apartado para subir los videos siendo premium para subirlos en diferentes calidades y marca si lo puede ver tanto usuarios free como premium



- Se muestran los dos botones 480p y 720p HD
- La etiqueta dirá Plan Premium



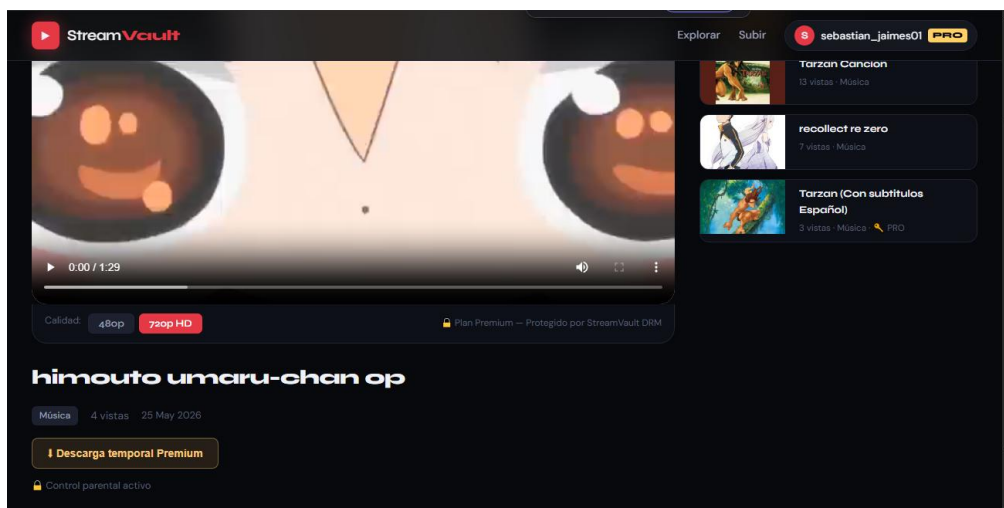
La vista de un usuario Free al intentar ver el mismo video:



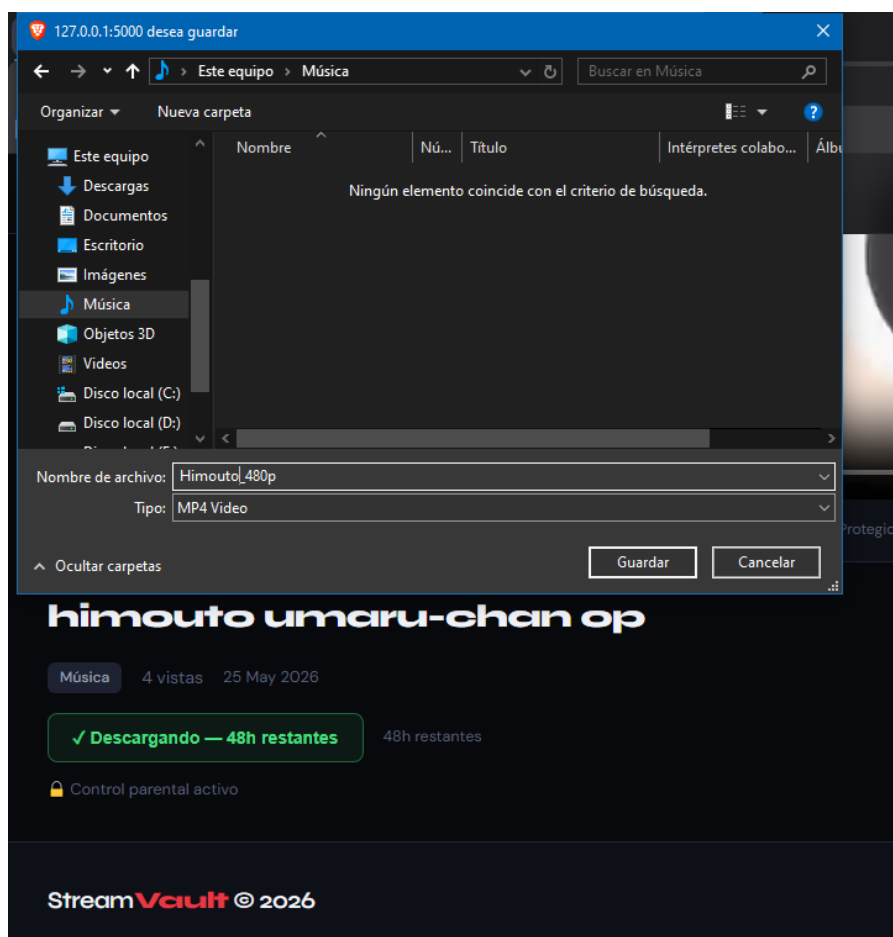
Patrón Proxy — Descarga temporal Premium

Los usuarios Premium pueden descargar contenido temporalmente y disfrutarlo incluso sin conexión a internet. Esta característica ofrece mayor flexibilidad y comodidad, ideal para viajes, desplazamientos o lugares con acceso limitado a la red.

Entrar a cualquier video que se quiera descargar



Le da en descargar y puedes elegir en donde se guardara



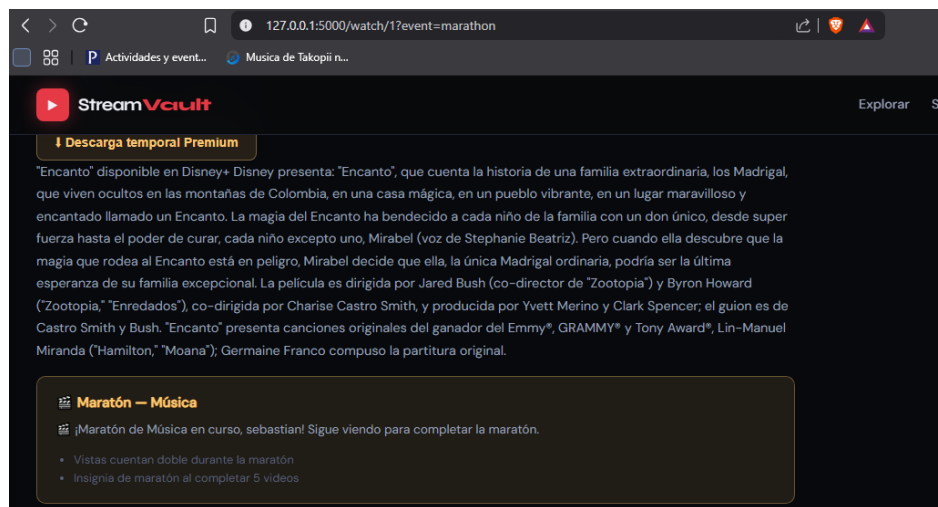
Template Method — Eventos especiales de visualización

La plataforma va más allá de la reproducción tradicional de videos. StreamVault incorpora eventos especiales como maratones, estrenos exclusivos y transmisiones en vivo, creando experiencias únicas que aumentan la interacción de los usuarios y generan mayor expectativa alrededor del contenido.

Esto simula ciertos eventos que puede tener la aplicación utilizando

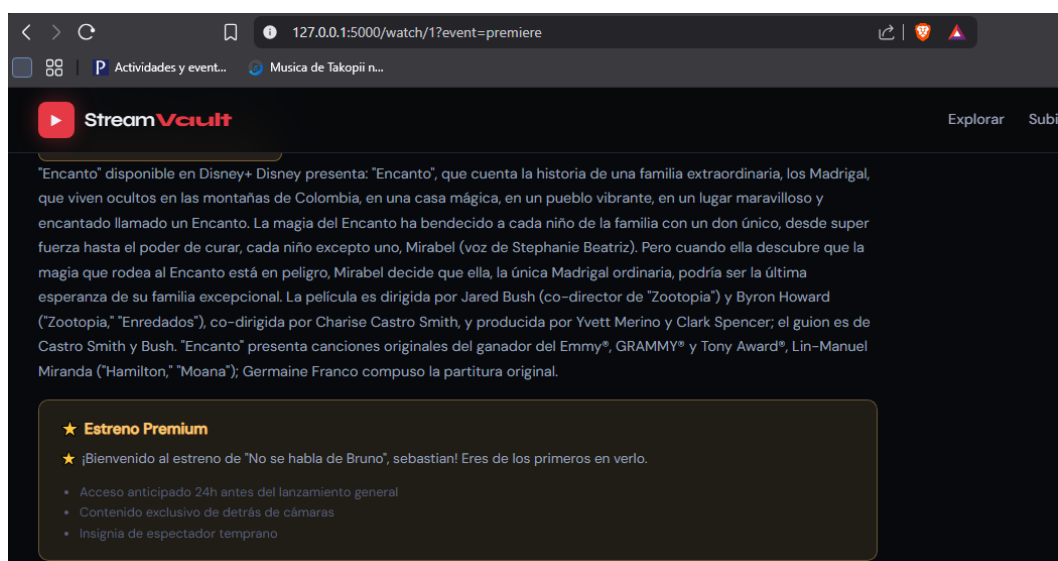
Maratón — disponible para todos los usuarios

<http://127.0.0.1:5000/watch/1?event=marathon>



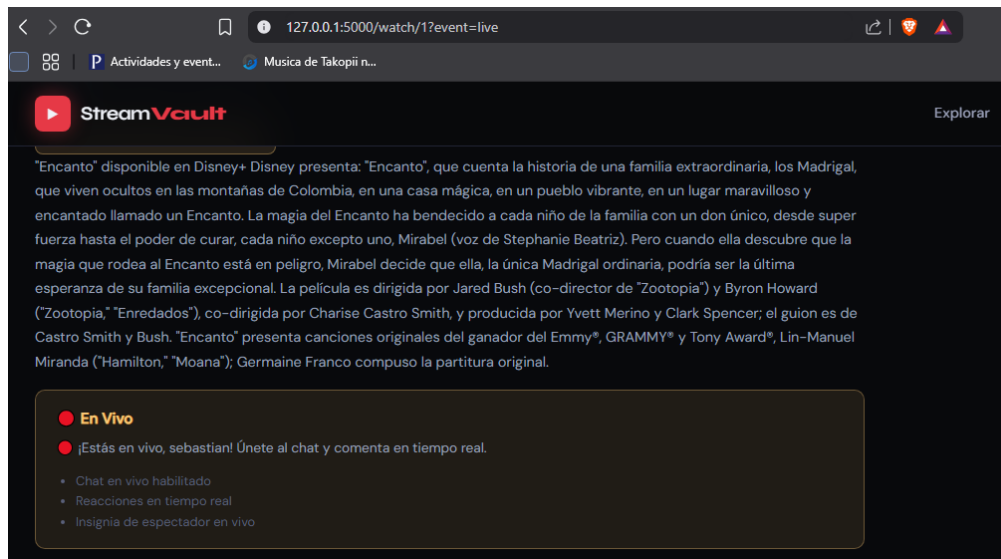
Estreno Premium — solo para usuarios Premium

<http://127.0.0.1:5000/watch/1?event=premiere>



En Vivo — disponible para todos

<http://127.0.0.1:5000/watch/1?event=live>

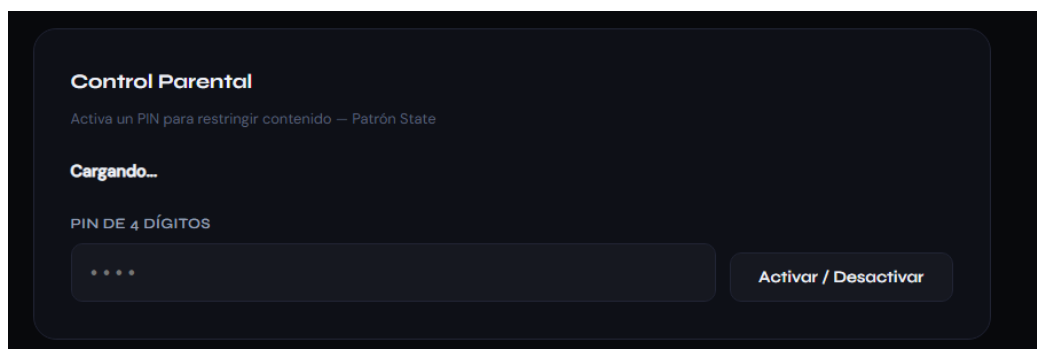


Objetivo 4 - Seguridad

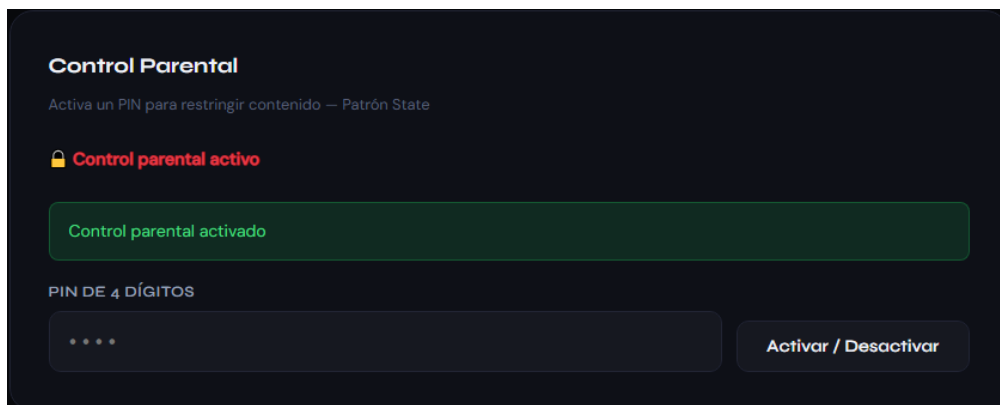
Patrón State — Control parental con PIN

La seguridad familiar es una prioridad. StreamVault incorpora un sistema de control parental que permite restringir contenido sensible mediante un PIN de seguridad, ofreciendo tranquilidad a padres y tutores y garantizando una experiencia adecuada para usuarios de todas las edades.

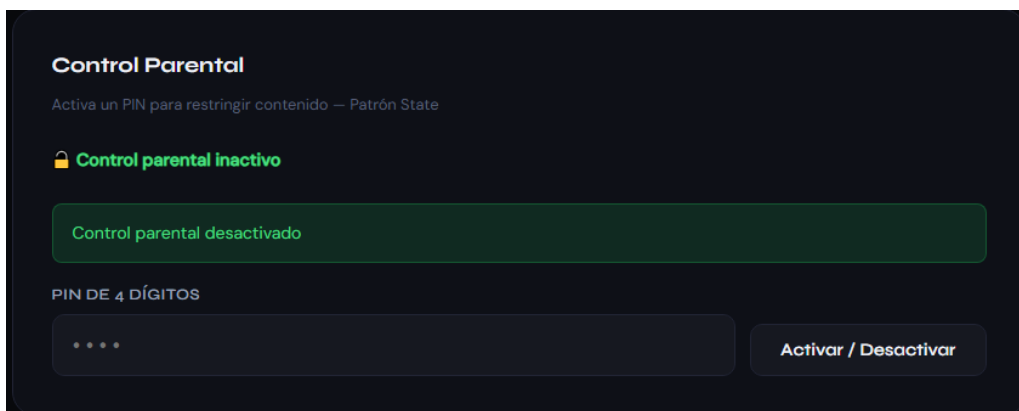
Se debe activar el control parental



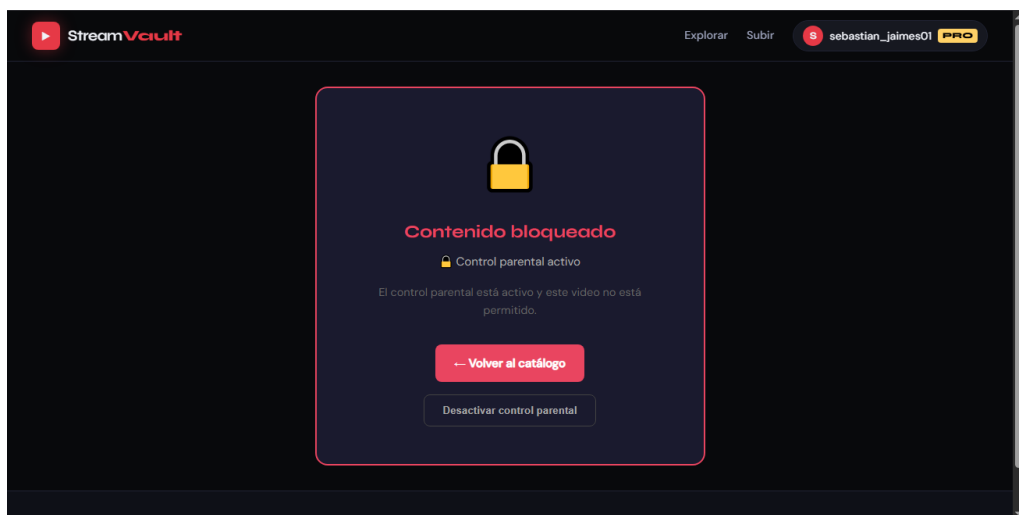
Vista de cuando esta activo



Vista de cuando se desactiva



Cuando entras a un video para +18 con el control parental activo sale esta pantalla

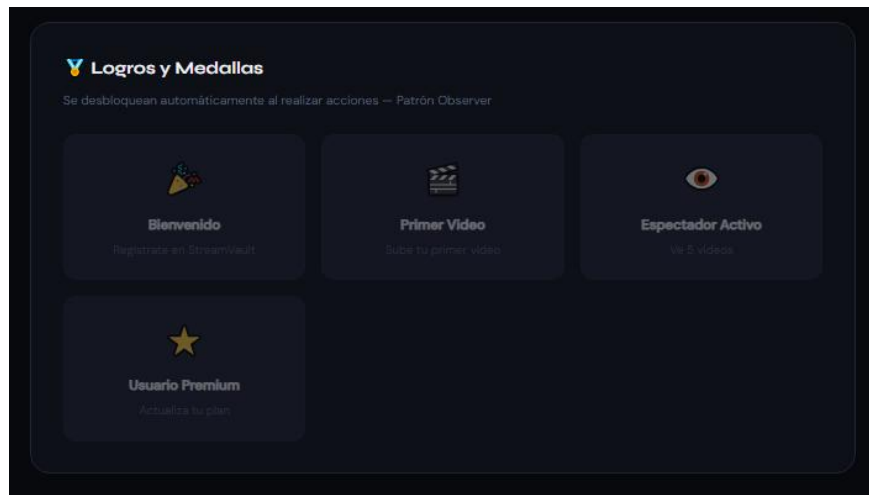
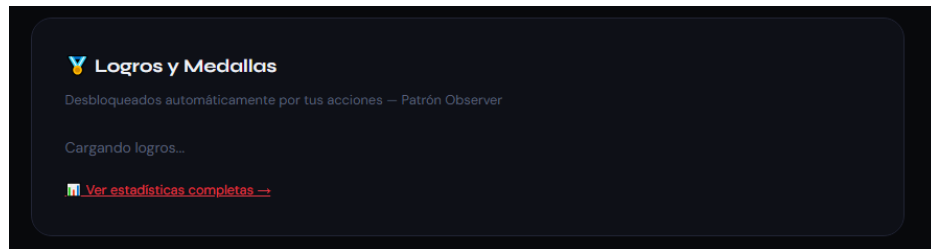


Objetivo 5 - Fidelización

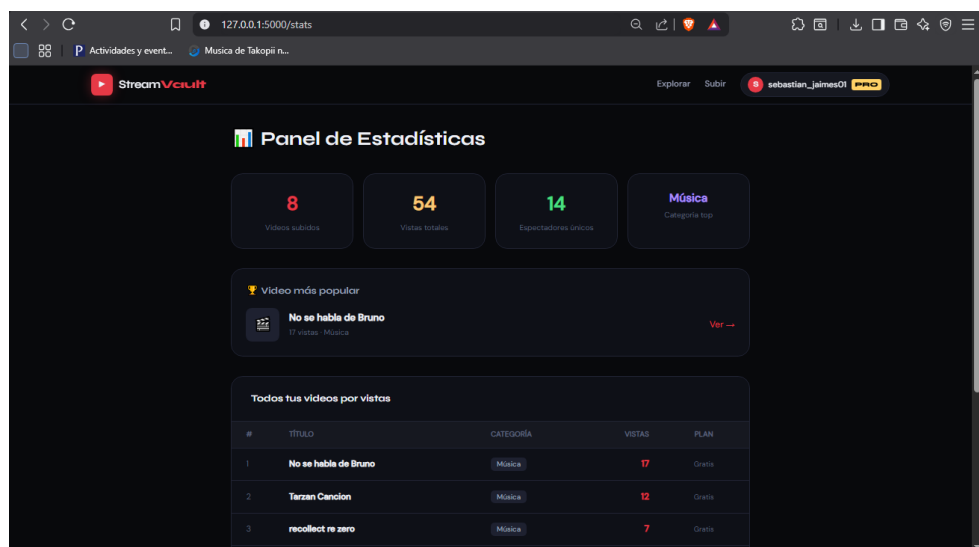
Patrón Observer — Sistema de logros y medallas

La plataforma transforma la experiencia de visualización en una actividad más dinámica y entretenida. Los usuarios pueden desbloquear logros, obtener medallas y seguir su progreso dentro de la aplicación, incentivando la participación constante y fortaleciendo el vínculo con la plataforma.

Apartado para los logros y medallas



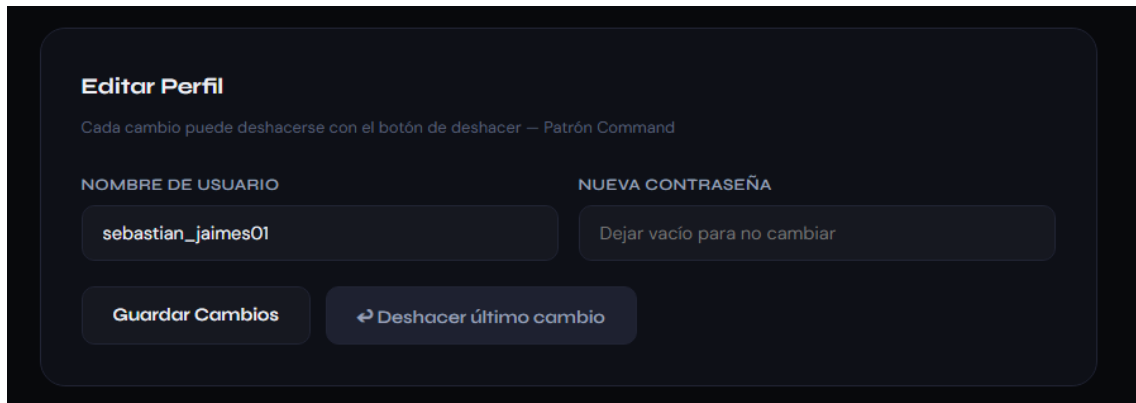
Aquí se ven las estadísticas de tus videos subidos



Objetivo 6 - Recuperación y respaldo

Patrón Command — Cambios de perfil con deshacer

En StreamVault entendemos que los errores pueden ocurrir. Por ello, los usuarios cuentan con la posibilidad de revertir cambios realizados en su perfil, recuperando información anterior de manera rápida y sencilla. Esta funcionalidad brinda confianza y seguridad al momento de personalizar la cuenta.



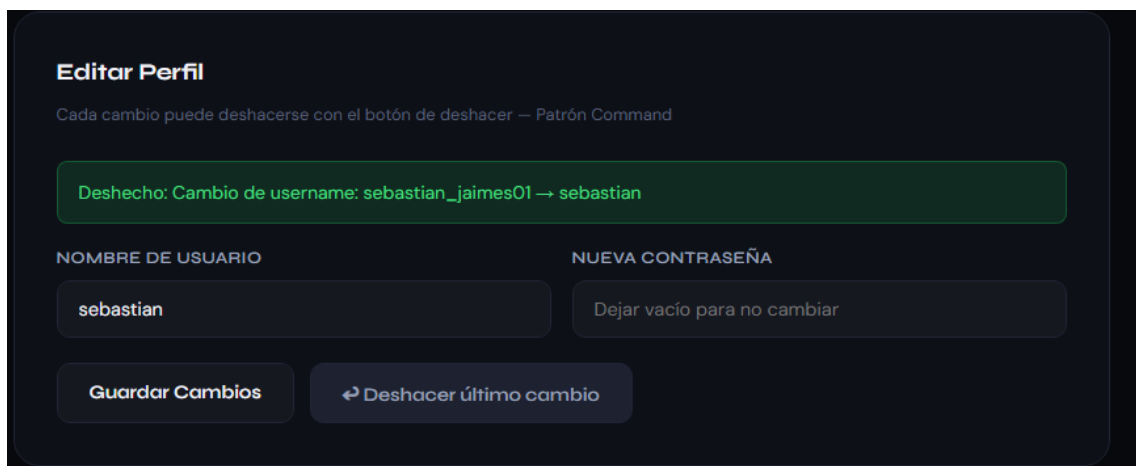
Editar Perfil

Cada cambio puede deshacerse con el botón de deshacer — Patrón Command

NOMBRE DE USUARIO: sebastian_jaimes01

NUEVA CONTRASEÑA: Dejar vacío para no cambiar

Guardar Cambios ↩️ Deshacer último cambio



Editar Perfil

Cada cambio puede deshacerse con el botón de deshacer — Patrón Command

Deshecho: Cambio de username: sebastian_jaimes01 → sebastian

NOMBRE DE USUARIO: sebastian

NUEVA CONTRASEÑA: Dejar vacío para no cambiar

Guardar Cambios ↩️ Deshacer último cambio

Memento - Borradores de formulario

Pensando en la comodidad de los creadores de contenido, StreamVault guarda automáticamente los datos ingresados en los formularios de publicación. De esta manera, si ocurre un cierre inesperado o una interrupción, el usuario puede continuar exactamente donde se quedó sin perder su trabajo.

Se guarda tanto el título como la categoría y la descripción de video por si se cierra la sesión y se quiere volver a retomar

StreamVault

Explorar

Subir

8

sebastian

PRO

↑ Subir Video

Comparte tu contenido con la comunidad

TÍTULO DEL VIDEO *

CATEGORÍA *

Prueba de borrador 1

Deportes

DESCRIPCIÓN

Pruebas

StreamVault

Explorar

Subir

8

sebastian

PRO

Solo formato .vtt — WebVTT

■

CONTENIDO PREMIUM (SOLO PARA SUSCRIPTORES)

■

🔞

CONTENIDO PARA MAYORES DE 18 AÑOS

SERÁ BLOQUEADO POR EL CONTROL PARENTAL — PATRÓN STATE

↑ Subir Video

Borradores

Patrón Memento

Guardar borrador

Borrador 2:25:36 p.m. 01 Jun 2026 19:25

prueba · Películas

Restaurar

Patrones eliminados	Razon
Prototype	Se eliminó porque la funcionalidad de duplicar videos no formaba parte de los requerimientos reales del proyecto. La ruta creada para aplicar el patrón no era utilizada por el frontend, convirtiéndose en código muerto que añadía complejidad sin aportar valor.
Adapter	Se descartó porque el sistema nunca implementó una integración real con Google OAuth. El único método de autenticación funcional era usuario y contraseña, por lo que el Adapter simulaba una funcionalidad inexistente.
Bridge	Se eliminó porque las notificaciones solo se mostraban mediante print() en la consola del servidor y nunca llegaban al usuario. Además, quedó oculto dentro de la Facade de registro, perdiendo visibilidad y utilidad arquitectónica.

Patrón Command

El problema que había antes

La ruta /api/profile modificaba el usuario directamente en la base de datos sin guardar el estado anterior. Si el usuario cambiaba su username por error no había forma de recuperar el anterior.

Antes:

if 'username' in data:

 user.username = data['username']

if 'password' in data and data['password']:

 user.set_password(data['password'])

db.session.commit()

Despues:

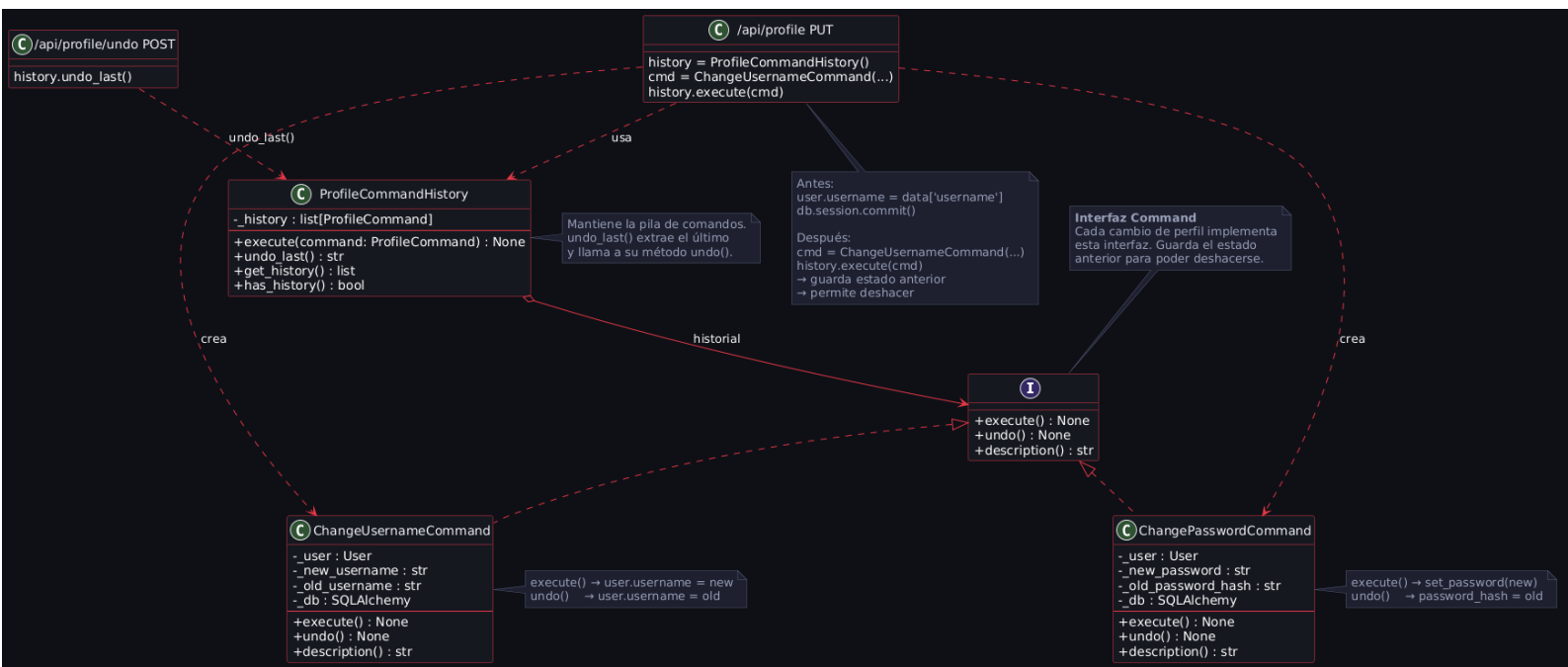
```
@app.route('/api/profile', methods=['PUT'])
@login_required
def update_profile():
    user = get_current_user()
    data = request.get_json()

    # — Command —
    # Cada cambio de perfil es un Command que guarda el estado anterior
    # y puede deshacerse. El historial permite revertir el último cambio.
    history = profile_histories.setdefault(user.id, ProfileCommandHistory())

    if 'username' in data and data['username']:
        existing = User.query.filter_by(username=data['username']).first()
        if existing and existing.id != user.id:
            return jsonify(
                APIResponseBuilder().set_error('Username taken').build()), 400
        cmd = ChangeUsernameCommand(user, data['username'], db)
        history.execute(cmd)

    if 'password' in data and data['password']:
        cmd = ChangePasswordCommand(user, data['password'], db)
        history.execute(cmd)

    #
```



Patrón State

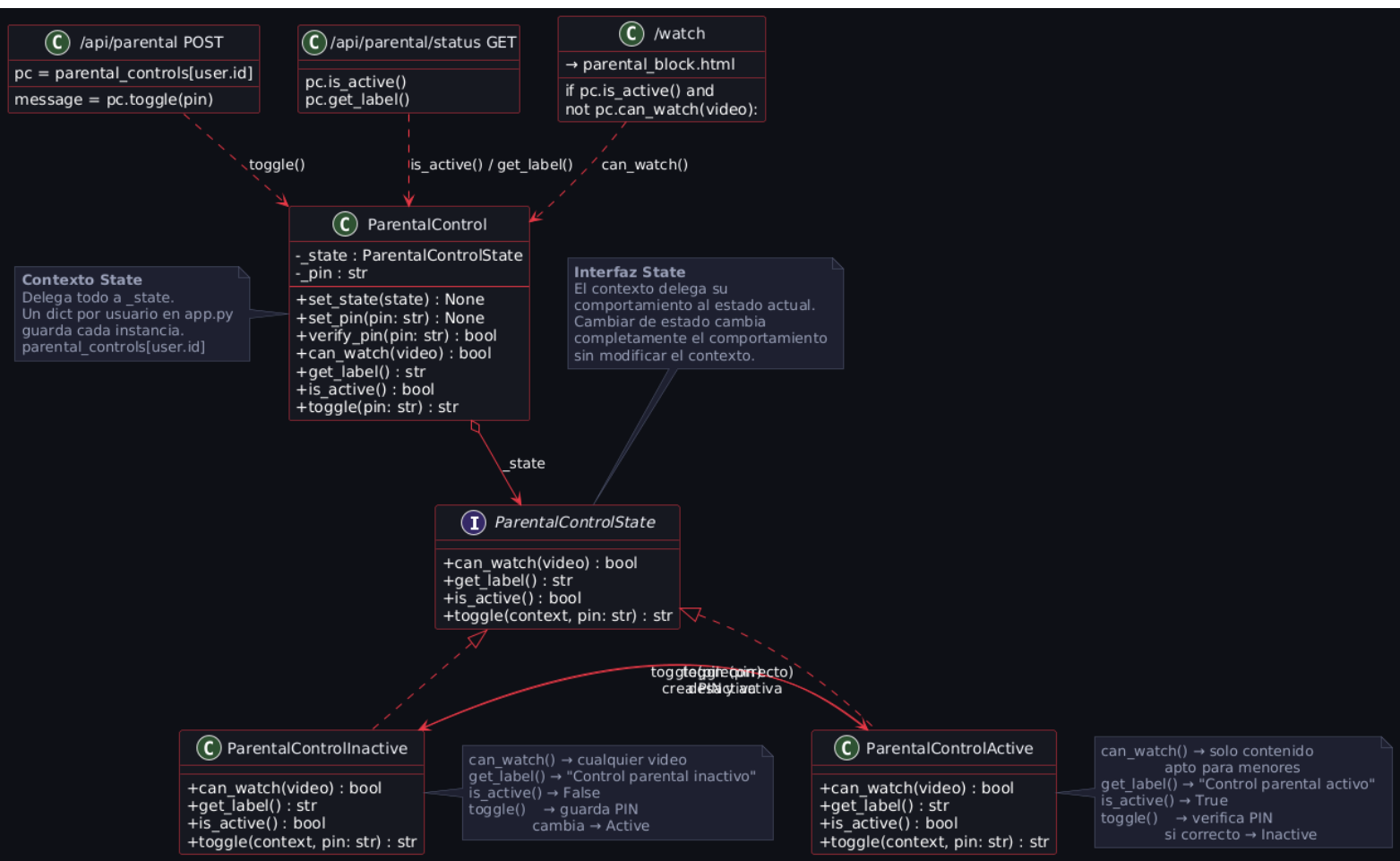
El problema que había antes

No existía control parental. Sin el patrón, para implementarlo habría que agregar un campo booleano `parental_active` al usuario y verificarlo con `if user.parental_active` en cada ruta que sirva contenido. Activar y desactivar requeriría lógica condicional en múltiples lugares.

Antes no existía

```
if video.is_premium and not access.can_watch_premium():
    return render_template('upgrade.html', user=user, video=video)

# — State — control parental —————
pc = parental_controls.get(user.id)
if pc and pc.is_active() and not pc.can_watch(video):
    return render_template('parental_block.html', user=user,
                           label=pc.get_label())
#
```



Patrón Observer

El problema que había antes

No existía ningún sistema de logros. Para agregar uno sin el patrón, cada ruta tendría que saber qué logros existen y verificar manualmente si el usuario los cumple. Agregar un logro nuevo significaría modificar múltiples rutas.

Antes:

Registro sin notificación

```
user = UserFactory.create(...)
```

```
db.session.add(user)
```

```
db.session.commit()
```

```
session['user_id'] = user.id
```

```
return jsonify({'success': True, 'redirect': '/'})
```

```
facade = UserRegistrationFacade(db, session, User)
result = facade.register(username=username, email=email, password=password)
# -----

if not result.success:
    return jsonify(APIResponseBuilder().set_error(result.error).build()), 400

# — Observer —
achievements = achievement_system.notify('user_registered', result.user)
# -----
```



Patrón Strategy

El problema que había antes

La ruta `/api/recommendations` siempre usaba el mismo algoritmo fijo — combinaba videos de categorías favoritas con los más vistos. No había forma de cambiar el algoritmo sin modificar directamente el código de la ruta.

Antes:

```
cat_videos = Video.query.filter(Video.category.in_(fav_cats))
```

```
.order_by(Video.views.desc()).limit(6).all()
```

```
popular = Video.query.order_by(Video.views.desc()).limit(6).all()
```

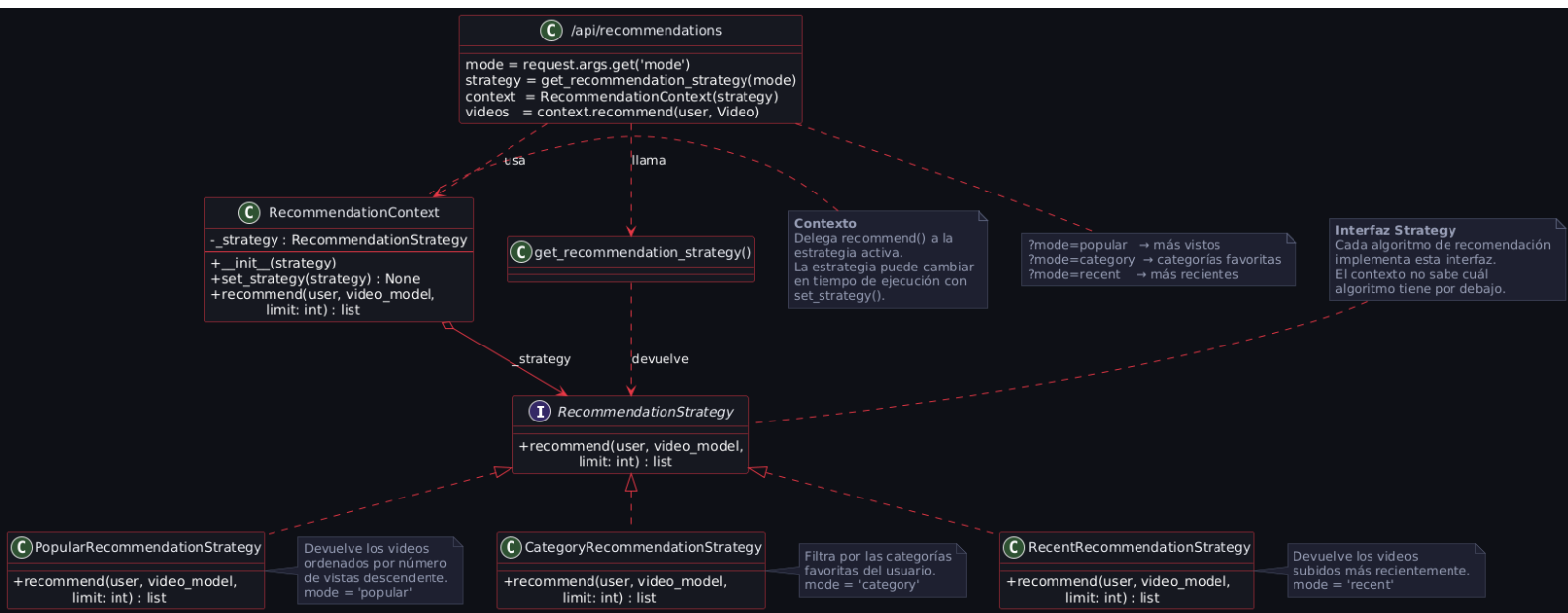
```
combined = cat_videos + [v for v in popular if v.id not in seen_ids]
```

```
return jsonify({'videos': combined})
```

Despues:

```
@app.route('/api/recommendations')
@login_required
def recommendations():
    user = get_current_user()
    mode = request.args.get('mode', 'popular')

    # — Strategy —
    # Selecciona el algoritmo de recomendación según el parámetro mode.
    # popular → videos más vistos
    # category → categorías favoritas del usuario
    # recent → videos más recientes
    strategy = get_recommendation_strategy(mode)
    context = RecommendationContext(strategy)
    videos = context.recommend(user, Video, limit=12)
    #
```



Patrón Proxy

El problema que había antes

No existía ningún sistema de descarga. Sin el Proxy, para implementar descargas habría que dar acceso directo a los archivos de video a cualquier usuario que conociera la URL, sin verificar si tiene plan Premium ni controlar por cuánto tiempo puede acceder.

Antes — acceso directo sin control:

python

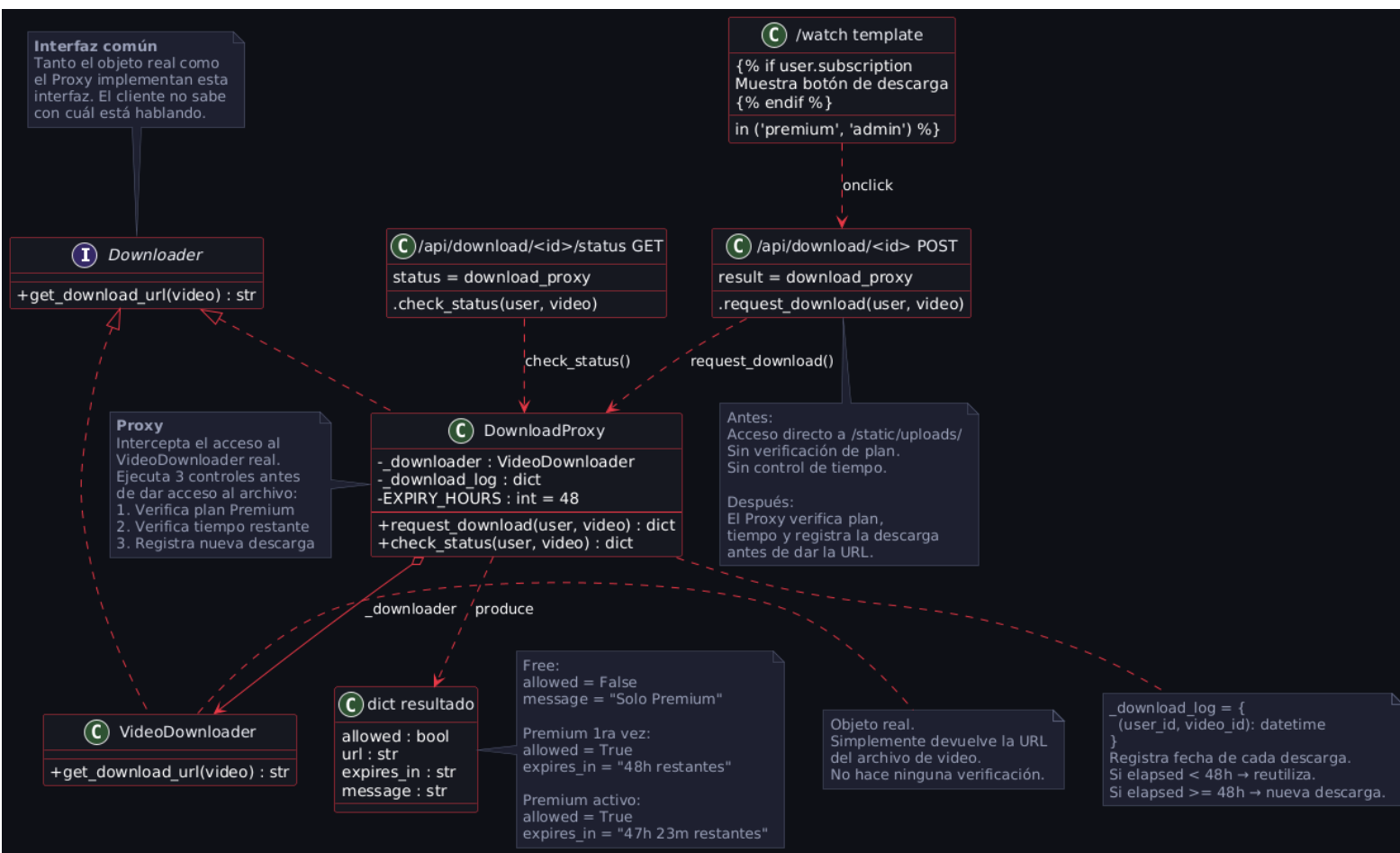
Cualquiera podía acceder al archivo directamente

<http://127.0.0.1:5000/static/uploads/archivo.mp4>

Sin verificar plan, sin tiempo límite, sin registro

Después con el Proxy:

```
{% if user.subscription in ('premium', 'admin') %}
<!-- Proxy - Descarga temporal Premium -->
<div id="download-section" style="margin-top:1rem; display:flex; align-items:center; gap:1rem; flex-wrap:wrap;">
  <button id="download-btn"
    onclick="requestDownload({{ video.id }})"
    style="display:inline-flex; align-items:center; gap:0.5rem; background: rgba(250,199,117,0.12); border: 1px solid #ccc; padding: 5px 10px;">
    Descarga temporal Premium
  </button>
  <span id="download-timer" style="font-size:0.78rem; color:var(--text-3);"></span>
</div>
{% endif %}
```



Patron Composite

El problema que había antes

La biblioteca personal era una lista plana de videos sin ninguna organización. No había forma de agrupar videos por tema, crear colecciones personalizadas ni ver estadísticas agrupadas. Todos los videos aparecían mezclados sin estructura.

Antes:

La biblioteca era solo una lista plana

lib._video_ids = [1, 2, 3, 4, 5]

Sin agrupación, sin estadísticas por grupo

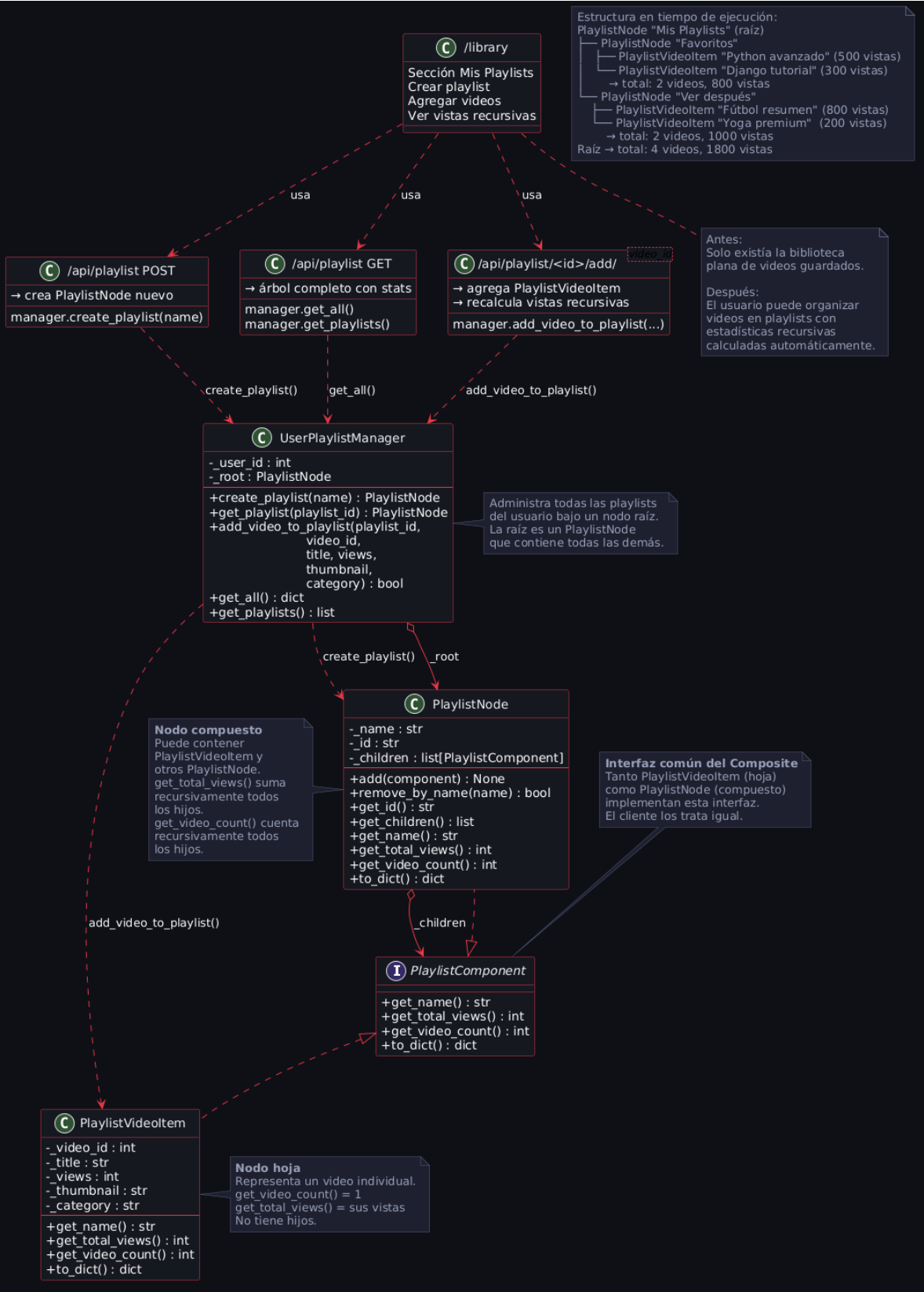
Después:

```
# — Composite — crear nodo playlist —
manager = get_playlist_manager(user.id)
playlist = manager.create_playlist(name)
#
```

```
# — Composite — agregar hoja al nodo —
manager = get_playlist_manager(user.id)
success = manager.add_video_to_playlist(
    playlist_id = playlist_id,
    video_id     = video.id,
    title        = video.title,
    views        = video.views,
    thumbnail    = video.thumbnail,
    category     = video.category
)
#
```

```
return jsonify(APIResponseBuilder().set_error('Playlist no encontrada').build()), 404

playlist = manager.get_playlist(playlist_id)
response = APIResponseBuilder().set_success(True).set_message('Video agregado a la playlist').build()
response['video_count'] = playlist.get_video_count()
response['total_views'] = playlist.get_total_views()
return jsonify(response)
```



Template Method

El problema que había antes

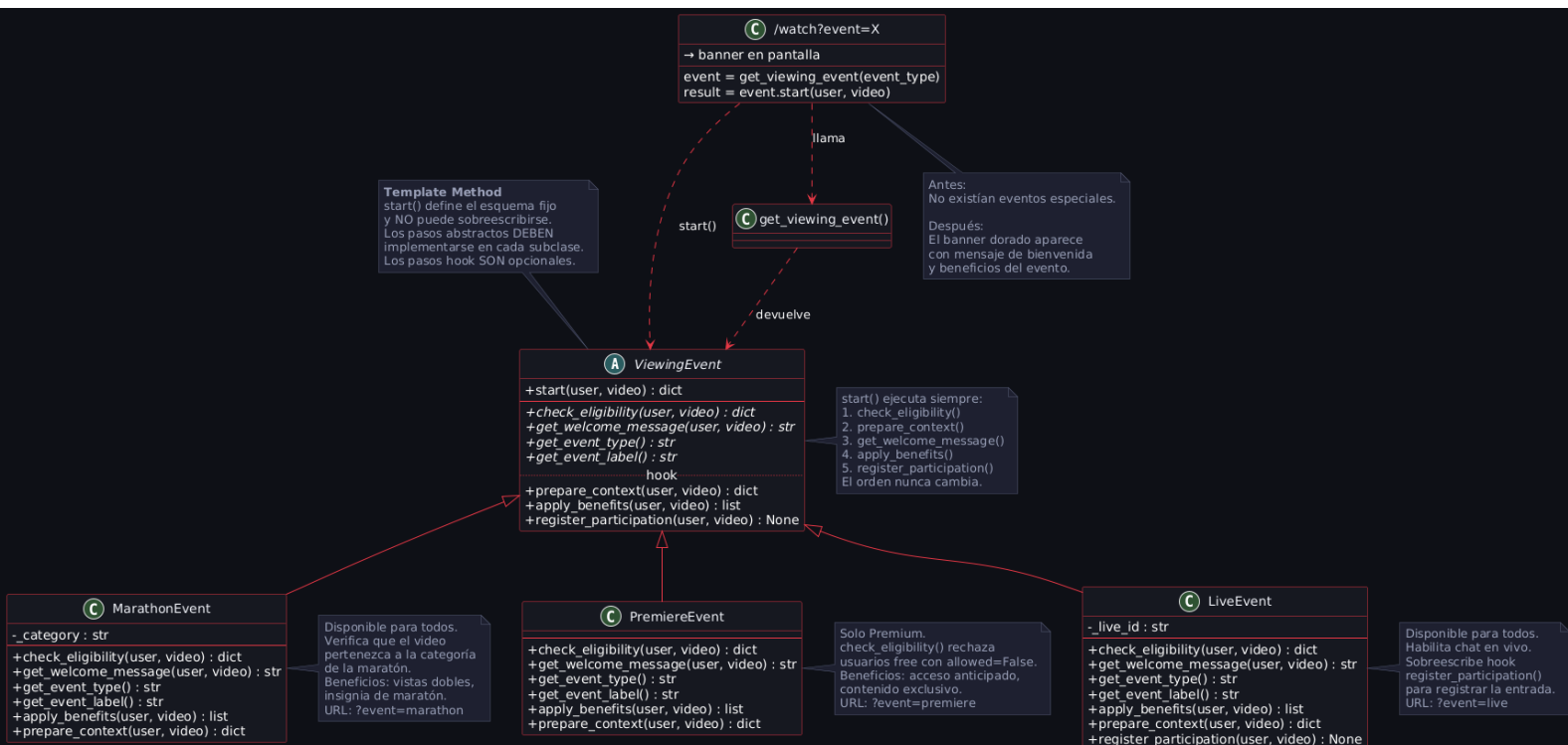
La ruta /watch no tenía ningún sistema de eventos especiales. Todos los usuarios veían el mismo reproductor sin importar si era un estreno, una maratón o una transmisión en vivo. Para agregar distintos tipos de eventos habría que escribir múltiples bloques if/elif dentro de la ruta, mezclando la lógica de cada evento con el código del reproductor.

Antes:

```
@app.route('/watch/<int:video_id>')  
  
def watch(video_id):  
    user = get_current_user()  
    video = Video.query.get_or_404(video_id)  
    # No había eventos, todos veían lo mismo  
    return render_template('watch.html', user=user, video=video)
```

Después:

```
# — Template Method – evento especial de visualización —  
event_type = request.args.get('event')  
event_data = {}  
if event_type:  
    event = get_viewing_event(  
        event_type,  
        category=video.category,  
        live_id=request.args.get('live_id', 'live_1')  
    )  
    if event:  
        event_data = event.start(user, video)
```



Chain of Responsibility

El problema que había antes

La ruta de subida solo verificaba que el título y la categoría no estuvieran vacíos. No había validación de formato de archivo, longitud del título, categorías válidas ni contenido de la descripción. Todo eso quedaba sin verificar antes de guardar el video.

Antes:

python

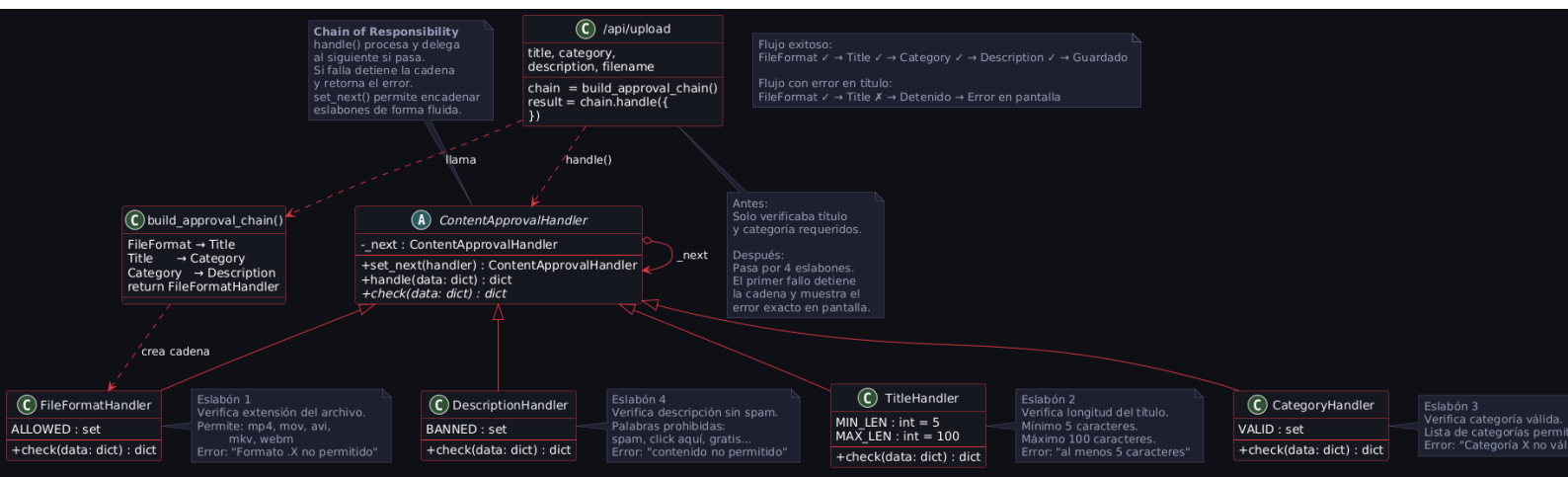
```
if not title or not category:
```

```
    return jsonify({'error': 'Título y categoría son requeridos'}), 400
```

El video se guardaba directamente sin más verificaciones

Después:

```
# — Chain of Responsibility — aprobación de contenido —————
# El contenido pasa por 4 eslabones antes de guardarse:
# FileFormat → Title → Category → Description
chain = build_approval_chain()
result = chain.handle({
    'title': title,
    'category': category,
    'description': description,
    'filename_480p': request.files.get('video_480p', type=lambda f: f.filename if f else ''),
})
if not result['approved']:
    return jsonify(
        APIResponseBuilder().set_error(result.get('reason', 'Contenido rechazado')).build()
    ), 400
```



Patron Memento

Memento 1 — Biblioteca personal

El problema que había antes

La biblioteca personal permitía agregar y eliminar videos pero no había forma de deshacer un cambio accidental. Si el usuario eliminaba un video por error, se perdía sin posibilidad de recuperarlo.

Antes:

python

No existía sistema de deshacer

lib.remove_video(video_id)

```
db.session.commit()
```

```
# Video eliminado permanentemente de la biblioteca
```

Después:

```
# — Memento — guarda estado antes de modificar —————
caretaker.backup(label=f'Antes de agregar video {video_id}')
lib.add_video(video_id)
# —————
```

```
# — Memento — guarda estado antes de eliminar —————
caretaker.backup(label=f'Antes de eliminar video {video_id}')
lib.remove_video(video_id)
# —————
```

Memento 2 — Borradores del formulario de subida

El problema que había antes

El formulario de subida no guardaba ningún estado. Si el usuario llenaba el título, descripción y categoría y cerraba el navegador accidentalmente, perdía todo y tenía que empezar desde cero.

Antes no existía

Después:

```
# — MEMENTO — borradores de formulario de subida —————
@app.route('/api/draft', methods=['POST'])
@login_required
def save_draft():
    """Guarda un borrador del formulario de subida."""
    user = get_current_user()
    data = request.get_json()

    # — Memento — guardar snapshot del formulario —————
    caretaker = get_draft_caretaker(user.id)
    caretaker.save_draft(
        title      = data.get('title', ''),
        description = data.get('description', ''),
        category    = data.get('category', ''),
        is_premium  = data.get('is_premium', False),
        label       = data.get('label', '')
    )
```

